

Competitive Programming and Time Complexity

Sean

April 27, 2026



Outline

- Competitive programming
- Time complexity

Introduction

- This lecture introduces what competitive programming is and how we analyze algorithm speed



Introduction

- This lecture introduces what competitive programming is and how we analyze algorithm speed
- The goal is to build good habits before learning more advanced algorithms and data structures



Introduction

- This lecture introduces what competitive programming is and how we analyze algorithm speed
- The goal is to build good habits before learning more advanced algorithms and data structures
- We will focus on problem-solving mindset first, then on time complexity



1 Competitive Programming

2 Time Complexity



Competitive Programming

Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits



Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits
- Each problem gives you a task, an input format, an output format, and constraints



Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits
- Each problem gives you a task, an input format, an output format, and constraints
- Your goal is to design a correct algorithm, implement it, and make it fast enough



Competitive Programming

- Competitive programming is about solving algorithmic problems under time and memory limits
- Each problem gives you a task, an input format, an output format, and constraints
- Your goal is to design a correct algorithm, implement it, and make it fast enough
- The solution is judged automatically on many hidden test cases



What a Problem Usually Contains

- Problem statement: what you need to compute



What a Problem Usually Contains

- Problem statement: what you need to compute
- Input format: how the data is given



What a Problem Usually Contains

- Problem statement: what you need to compute
- Input format: how the data is given
- Output format: what your program must print



What a Problem Usually Contains

- Problem statement: what you need to compute
- Input format: how the data is given
- Output format: what your program must print
- Constraints: how large the input can be

What a Problem Usually Contains

- Problem statement: what you need to compute
- Input format: how the data is given
- Output format: what your program must print
- Constraints: how large the input can be
- Sample tests: small examples to clarify the statement

A Good Solving Workflow

- 1 Understand the statement and rewrite the task in your own words



A Good Solving Workflow

- 1 Understand the statement and rewrite the task in your own words
- 2 Check the constraints and guess the needed complexity

A Good Solving Workflow

- 1 Understand the statement and rewrite the task in your own words
- 2 Check the constraints and guess the needed complexity
- 3 Find the algorithm or data structure

A Good Solving Workflow

- 1 Understand the statement and rewrite the task in your own words
- 2 Check the constraints and guess the needed complexity
- 3 Find the algorithm or data structure
- 4 Prove to yourself why the idea is correct



A Good Solving Workflow

- 1 Understand the statement and rewrite the task in your own words
- 2 Check the constraints and guess the needed complexity
- 3 Find the algorithm or data structure
- 4 Prove to yourself why the idea is correct
- 5 Implement carefully and test edge cases



Reading Constraints

- Constraints are not just technical details



Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible



Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible
- For example, checking all pairs may work for $n = 2000$



Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible
- For example, checking all pairs may work for $n = 2000$
- The same approach will usually fail for $n = 2 \times 10^5$



Reading Constraints

- Constraints are not just technical details
- They tell you which approaches are likely possible
- For example, checking all pairs may work for $n = 2000$
- The same approach will usually fail for $n = 2 \times 10^5$
- This is why time complexity is one of the first skills to learn



Common Judge Results

- AC: accepted, the solution passed all tests



Common Judge Results

- AC: accepted, the solution passed all tests
- WA: wrong answer, the program finished but produced incorrect output



Common Judge Results

- AC: accepted, the solution passed all tests
- WA: wrong answer, the program finished but produced incorrect output
- TLE: time limit exceeded, the program was too slow



Common Judge Results

- AC: accepted, the solution passed all tests
- WA: wrong answer, the program finished but produced incorrect output
- TLE: time limit exceeded, the program was too slow
- MLE: memory limit exceeded, the program used too much memory



Common Judge Results

- AC: accepted, the solution passed all tests
- WA: wrong answer, the program finished but produced incorrect output
- TLE: time limit exceeded, the program was too slow
- MLE: memory limit exceeded, the program used too much memory
- RE: runtime error, the program crashed during execution



Where to Practice

- CSES: excellent for structured beginner-to-intermediate practice



Where to Practice

- [CSES](#): excellent for structured beginner-to-intermediate practice
- [Codeforces](#): strong contest culture, frequent rounds, and many classic problems



Where to Practice

- [CSES](#): excellent for structured beginner-to-intermediate practice
- [Codeforces](#): strong contest culture, frequent rounds, and many classic problems
- [AtCoder](#): clean statements and very high-quality problem sets



Where to Practice

- [CSES](#): excellent for structured beginner-to-intermediate practice
- [Codeforces](#): strong contest culture, frequent rounds, and many classic problems
- [AtCoder](#): clean statements and very high-quality problem sets
- [USACO](#): great training path, especially for students building fundamentals step by step



Where to Practice

- [CSES](#): excellent for structured beginner-to-intermediate practice
- [Codeforces](#): strong contest culture, frequent rounds, and many classic problems
- [AtCoder](#): clean statements and very high-quality problem sets
- [USACO](#): great training path, especially for students building fundamentals step by step
- Different platforms feel different, so using more than one is helpful



How to Practice Effectively

- Solve problems regularly instead of cramming many problems in one day



How to Practice Effectively

- Solve problems regularly instead of cramming many problems in one day
- If you get stuck, think seriously first, then read the editorial and learn the key idea



How to Practice Effectively

- Solve problems regularly instead of cramming many problems in one day
- If you get stuck, think seriously first, then read the editorial and learn the key idea
- After reading a solution, try to re-implement it yourself without copying



How to Practice Effectively

- Solve problems regularly instead of cramming many problems in one day
- If you get stuck, think seriously first, then read the editorial and learn the key idea
- After reading a solution, try to re-implement it yourself without copying
- Revisit important problems later so the technique becomes familiar



How to Practice Effectively

- Solve problems regularly instead of cramming many problems in one day
- If you get stuck, think seriously first, then read the editorial and learn the key idea
- After reading a solution, try to re-implement it yourself without copying
- Revisit important problems later so the technique becomes familiar
- Keep track of patterns: prefix sums, greedy, binary search, graphs, DP, and data structures



Debugging Mindset

- When a solution fails, do not guess blindly; isolate the reason



Debugging Mindset

- When a solution fails, do not guess blindly; isolate the reason
- First decide whether the issue is correctness, time complexity, memory usage, or implementation detail



Debugging Mindset

- When a solution fails, do not guess blindly; isolate the reason
- First decide whether the issue is correctness, time complexity, memory usage, or implementation detail
- Test small cases, edge cases, and cases where you can compute the answer by hand



Debugging Mindset

- When a solution fails, do not guess blindly; isolate the reason
- First decide whether the issue is correctness, time complexity, memory usage, or implementation detail
- Test small cases, edge cases, and cases where you can compute the answer by hand
- Common bugs include off-by-one errors, wrong indexing, overflow, and forgetting special cases



Debugging Mindset

- When a solution fails, do not guess blindly; isolate the reason
- First decide whether the issue is correctness, time complexity, memory usage, or implementation detail
- Test small cases, edge cases, and cases where you can compute the answer by hand
- Common bugs include off-by-one errors, wrong indexing, overflow, and forgetting special cases
- A wrong answer is often easier to fix when you can produce one small counterexample



Helpful Contest Habits

- Start with a simple idea, then improve it if the constraints require it



Helpful Contest Habits

- Start with a simple idea, then improve it if the constraints require it
- Test small cases by hand before trusting the code



Helpful Contest Habits

- Start with a simple idea, then improve it if the constraints require it
- Test small cases by hand before trusting the code
- Pay attention to off-by-one errors and empty cases



Helpful Contest Habits

- Start with a simple idea, then improve it if the constraints require it
- Test small cases by hand before trusting the code
- Pay attention to off-by-one errors and empty cases
- Use clear variable names for important quantities



Helpful Contest Habits

- Start with a simple idea, then improve it if the constraints require it
- Test small cases by hand before trusting the code
- Pay attention to off-by-one errors and empty cases
- Use clear variable names for important quantities
- After solving, ask what pattern or technique the problem was teaching



Topics You Will See Later

- Basic data structures with STL
- Brute force and enumeration
- Searching algorithms
- Greedy algorithms
- Mathematics
- Graphs
- Dynamic programming
- Advanced data structures



1 Competitive Programming

2 Time Complexity



Time Complexity

Time Complexity

- Time complexity describes how the running time grows as the input size grows



Time Complexity

- Time complexity describes how the running time grows as the input size grows
- In competitive programming, we usually care more about growth rate than exact running time



Time Complexity

- Time complexity describes how the running time grows as the input size grows
- In competitive programming, we usually care more about growth rate than exact running time
- We use asymptotic notation such as $\mathcal{O}(1)$, $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, and $\mathcal{O}(n^2)$



Time Complexity

- Time complexity describes how the running time grows as the input size grows
- In competitive programming, we usually care more about growth rate than exact running time
- We use asymptotic notation such as $\mathcal{O}(1)$, $\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, and $\mathcal{O}(n^2)$
- The main goal is to decide whether an algorithm is fast enough for the given constraints



Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows



Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows
- In competitive programming, we usually focus more on time complexity, but memory limits still matter



Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows
- In competitive programming, we usually focus more on time complexity, but memory limits still matter
- Arrays, graphs, dynamic programming tables, and recursion depth can all affect memory usage



Space Complexity

- Space complexity describes how much memory an algorithm uses as the input size grows
- In competitive programming, we usually focus more on time complexity, but memory limits still matter
- Arrays, graphs, dynamic programming tables, and recursion depth can all affect memory usage
- A solution can fail even if it is fast enough when it uses too much memory



Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound



Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound
- $\Omega(f(n))$: an asymptotic lower bound



Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound
- $\Omega(f(n))$: an asymptotic lower bound
- $\Theta(f(n))$: a tight bound, meaning both upper and lower bound



Asymptotic Notation Reference

- $\mathcal{O}(f(n))$: an asymptotic upper bound
- $\Omega(f(n))$: an asymptotic lower bound
- $\Theta(f(n))$: a tight bound, meaning both upper and lower bound
- In competitive programming, $\mathcal{O}(\cdot)$ is used most often in practice



What Are We Counting?

- We estimate the number of basic operations in the program

What Are We Counting?

- We estimate the number of basic operations in the program
- Examples: assignment, comparison, addition, subtraction, multiplication, division, input, output



What Are We Counting?

- We estimate the number of basic operations in the program
- Examples: assignment, comparison, addition, subtraction, multiplication, division, input, output
- A single basic operation is treated as $\mathcal{O}(1)$



What Are We Counting?

- We estimate the number of basic operations in the program
- Examples: assignment, comparison, addition, subtraction, multiplication, division, input, output
- A single basic operation is treated as $\mathcal{O}(1)$
- Then we study how many times loops and repeated operations run



Example: $\mathcal{O}(n)$

```
1  for (int i = 1; i <= n; ++i) {  
2      cout << i << '\n';  
3  }
```

Example: $\mathcal{O}(n)$

```
1  for (int i = 1; i <= n; ++i) {  
2      cout << i << '\n';  
3  }
```

- The loop runs n times

Example: $\mathcal{O}(n)$

```
1  for (int i = 1; i <= n; ++i) {  
2      cout << i << '\n';  
3  }
```

- The loop runs n times
- Each iteration does only a constant amount of work



Example: $\mathcal{O}(n)$

```
1  for (int i = 1; i <= n; ++i) {  
2      cout << i << '\n';  
3  }
```

- The loop runs n times
- Each iteration does only a constant amount of work
- So the total complexity is $\mathcal{O}(n)$



Example: Nested Loops

```
1  for (int i = 1; i <= n; ++i) {
2      for (int j = 1; j <= n; ++j) {
3          cout << i << ' ' << j << '\n';
4      }
5  }
6
7  for (int i = 1; i <= n; ++i) {
8      cout << i << '\n';
9  }
```



Example: Nested Loops

```
1  for (int i = 1; i <= n; ++i) {
2      for (int j = 1; j <= n; ++j) {
3          cout << i << ' ' << j << '\n';
4      }
5  }
6
7  for (int i = 1; i <= n; ++i) {
8      cout << i << '\n';
9  }
```

- The nested loops take $\mathcal{O}(n^2)$



Example: Nested Loops

```
1  for (int i = 1; i <= n; ++i) {
2      for (int j = 1; j <= n; ++j) {
3          cout << i << ' ' << j << '\n';
4      }
5  }
6
7  for (int i = 1; i <= n; ++i) {
8      cout << i << '\n';
9  }
```

- The nested loops take $\mathcal{O}(n^2)$
- The final single loop takes $\mathcal{O}(n)$



Example: Nested Loops

```
1  for (int i = 1; i <= n; ++i) {  
2      for (int j = 1; j <= n; ++j) {  
3          cout << i << ' ' << j << '\n';  
4      }  
5  }  
6  
7  for (int i = 1; i <= n; ++i) {  
8      cout << i << '\n';  
9  }
```

- The nested loops take $\mathcal{O}(n^2)$
- The final single loop takes $\mathcal{O}(n)$
- Together this is $\mathcal{O}(n^2 + n)$



Simplifying Big-O

- In Big-O, we ignore lower-order terms



Simplifying Big-O

- In Big-O, we ignore lower-order terms
- So $\mathcal{O}(n^2 + n)$ becomes $\mathcal{O}(n^2)$



Simplifying Big-O

- In Big-O, we ignore lower-order terms
- So $\mathcal{O}(n^2 + n)$ becomes $\mathcal{O}(n^2)$
- We also ignore constant factors, so $\mathcal{O}(2n^2)$ is still $\mathcal{O}(n^2)$



Simplifying Big-O

- In Big-O, we ignore lower-order terms
- So $\mathcal{O}(n^2 + n)$ becomes $\mathcal{O}(n^2)$
- We also ignore constant factors, so $\mathcal{O}(2n^2)$ is still $\mathcal{O}(n^2)$
- This is reasonable because when n is large, the highest-order term dominates the running time



Worst Case and Amortized Analysis

- Usually we analyze the worst case

Worst Case and Amortized Analysis

- Usually we analyze the worst case
- That is why Big-O often means an upper bound on the running time



Worst Case and Amortized Analysis

- Usually we analyze the worst case
- That is why Big-O often means an upper bound on the running time
- Sometimes most operations are $\mathcal{O}(1)$, but a few rare operations are $\mathcal{O}(n)$



Worst Case and Amortized Analysis

- Usually we analyze the worst case
- That is why Big-O often means an upper bound on the running time
- Sometimes most operations are $\mathcal{O}(1)$, but a few rare operations are $\mathcal{O}(n)$
- If we average the total cost over many operations, the result may still be $\mathcal{O}(1)$ amortized



Worst Case and Amortized Analysis

- Usually we analyze the worst case
- That is why Big-O often means an upper bound on the running time
- Sometimes most operations are $\mathcal{O}(1)$, but a few rare operations are $\mathcal{O}(n)$
- If we average the total cost over many operations, the result may still be $\mathcal{O}(1)$ amortized
- A common example is repeated `push_back` on a vector



Common Complexities

- $\mathcal{O}(1)$: basic operations, direct access



Common Complexities

- $\mathcal{O}(1)$: basic operations, direct access
- $\mathcal{O}(\log n)$: binary search, repeated halving



Common Complexities

- $\mathcal{O}(1)$: basic operations, direct access
- $\mathcal{O}(\log n)$: binary search, repeated halving
- $\mathcal{O}(n)$: one full pass over an array or string



Common Complexities

- $\mathcal{O}(1)$: basic operations, direct access
- $\mathcal{O}(\log n)$: binary search, repeated halving
- $\mathcal{O}(n)$: one full pass over an array or string
- $\mathcal{O}(n \log n)$: merge sort, many efficient divide-and-conquer algorithms



Common Complexities

- $\mathcal{O}(1)$: basic operations, direct access
- $\mathcal{O}(\log n)$: binary search, repeated halving
- $\mathcal{O}(n)$: one full pass over an array or string
- $\mathcal{O}(n \log n)$: merge sort, many efficient divide-and-conquer algorithms
- $\mathcal{O}(n^2)$: checking all pairs, many simple quadratic algorithms



Common Complexities

- $\mathcal{O}(1)$: basic operations, direct access
- $\mathcal{O}(\log n)$: binary search, repeated halving
- $\mathcal{O}(n)$: one full pass over an array or string
- $\mathcal{O}(n \log n)$: merge sort, many efficient divide-and-conquer algorithms
- $\mathcal{O}(n^2)$: checking all pairs, many simple quadratic algorithms
- $\mathcal{O}(2^n)$ and $\mathcal{O}(n!)$: usually only possible for very small n



A Practical Rule of Thumb

- A common estimate is that around 10^8 simple operations per second is near the practical limit



A Practical Rule of Thumb

- A common estimate is that around 10^8 simple operations per second is near the practical limit
- This is only a rule of thumb, not an exact law



A Practical Rule of Thumb

- A common estimate is that around 10^8 simple operations per second is near the practical limit
- This is only a rule of thumb, not an exact law
- Different judges, machines, languages, and implementations all affect the real running time



A Practical Rule of Thumb

- A common estimate is that around 10^8 simple operations per second is near the practical limit
- This is only a rule of thumb, not an exact law
- Different judges, machines, languages, and implementations all affect the real running time
- Still, the constraints often tell you what kinds of solutions are realistic



Constraints Guide the Solution

- If n is around 2×10^5 , then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable



Constraints Guide the Solution

- If n is around 2×10^5 , then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable
- If n is around 2000, then $\mathcal{O}(n^2)$ may still pass



Constraints Guide the Solution

- If n is around 2×10^5 , then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable
- If n is around 2000, then $\mathcal{O}(n^2)$ may still pass
- If n is around 20, then exponential search may be possible



Constraints Guide the Solution

- If n is around 2×10^5 , then $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ is often acceptable
- If n is around 2000, then $\mathcal{O}(n^2)$ may still pass
- If n is around 20, then exponential search may be possible
- A major competitive programming skill is reading the constraints and guessing the intended complexity



Optimizing an Idea

- After learning how to count complexity, the next step is to improve it

Optimizing an Idea

- After learning how to count complexity, the next step is to improve it
- Many algorithms and data structures are useful because they reduce the number of repeated operations



Optimizing an Idea

- After learning how to count complexity, the next step is to improve it
- Many algorithms and data structures are useful because they reduce the number of repeated operations
- A simple and classic example is prefix sums



Prefix Sum Example

CSES: Static Range Sum Queries

Given an array of length N and Q queries, each query asks for the sum of the subarray $[a, b]$, that is, $\sum_{i=a}^b x_i$. Assume $N, Q \leq 2 \times 10^5$.



Prefix Sum Example

CSES: Static Range Sum Queries

Given an array of length N and Q queries, each query asks for the sum of the subarray $[a, b]$, that is, $\sum_{i=a}^b x_i$. Assume $N, Q \leq 2 \times 10^5$.

- A brute-force query may sum up to N elements



Prefix Sum Example

CSES: Static Range Sum Queries

Given an array of length N and Q queries, each query asks for the sum of the subarray $[a, b]$, that is, $\sum_{i=a}^b x_i$. Assume $N, Q \leq 2 \times 10^5$.

- A brute-force query may sum up to N elements
- Doing that for all Q queries gives $\mathcal{O}(NQ)$, which is too slow



Prefix Sum Example

CSES: Static Range Sum Queries

Given an array of length N and Q queries, each query asks for the sum of the subarray $[a, b]$, that is, $\sum_{i=a}^b x_i$. Assume $N, Q \leq 2 \times 10^5$.

- A brute-force query may sum up to N elements
- Doing that for all Q queries gives $\mathcal{O}(NQ)$, which is too slow
- The problem is that many overlapping ranges are recomputed again and again



Prefix Sum Optimization

- Define $pre[i]$ as the sum of the first i elements:

$$pre_i = \sum_{j=1}^i x_j$$

Prefix Sum Optimization

- Define $pre[i]$ as the sum of the first i elements:

$$pre_i = \sum_{j=1}^i x_j$$

- Then the sum of $[a, b]$ is:

$$\sum_{i=a}^b x_i = pre_b - pre_{a-1}$$



Prefix Sum Optimization

- Define $pre[i]$ as the sum of the first i elements:

$$pre_i = \sum_{j=1}^i x_j$$

- Then the sum of $[a, b]$ is:

$$\sum_{i=a}^b x_i = pre_b - pre_{a-1}$$

- Building the prefix sum array takes $\mathcal{O}(N)$



Prefix Sum Optimization

- Define $pre[i]$ as the sum of the first i elements:

$$pre_i = \sum_{j=1}^i x_j$$

- Then the sum of $[a, b]$ is:

$$\sum_{i=a}^b x_i = pre_b - pre_{a-1}$$

- Building the prefix sum array takes $\mathcal{O}(N)$
- Each query is answered in $\mathcal{O}(1)$



Prefix Sum Optimization

- Define $pre[i]$ as the sum of the first i elements:

$$pre_i = \sum_{j=1}^i x_j$$

- Then the sum of $[a, b]$ is:

$$\sum_{i=a}^b x_i = pre_b - pre_{a-1}$$

- Building the prefix sum array takes $\mathcal{O}(N)$
- Each query is answered in $\mathcal{O}(1)$
- Total complexity becomes $\mathcal{O}(N + Q)$



2D Prefix Sum Example

CSES: Forest Queries

Given an $n \times n$ forest grid, each square is empty or contains a tree. Each query asks how many trees are inside a rectangle (y_1, x_1) to (y_2, x_2) . Assume $n \leq 1000$ and $q \leq 2 \times 10^5$.

2D Prefix Sum Example

CSES: Forest Queries

Given an $n \times n$ forest grid, each square is empty or contains a tree. Each query asks how many trees are inside a rectangle (y_1, x_1) to (y_2, x_2) . Assume $n \leq 1000$ and $q \leq 2 \times 10^5$.

- Brute force may scan up to n^2 cells per query



2D Prefix Sum Example

CSES: Forest Queries

Given an $n \times n$ forest grid, each square is empty or contains a tree. Each query asks how many trees are inside a rectangle (y_1, x_1) to (y_2, x_2) . Assume $n \leq 1000$ and $q \leq 2 \times 10^5$.

- Brute force may scan up to n^2 cells per query
- Total complexity can be $\mathcal{O}(n^2q)$, which is too slow



2D Prefix Sum Example

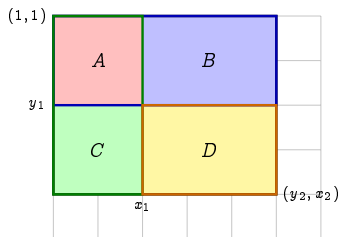
CSES: Forest Queries

Given an $n \times n$ forest grid, each square is empty or contains a tree. Each query asks how many trees are inside a rectangle (y_1, x_1) to (y_2, x_2) . Assume $n \leq 1000$ and $q \leq 2 \times 10^5$.

- Brute force may scan up to n^2 cells per query
- Total complexity can be $\mathcal{O}(n^2q)$, which is too slow
- We want to preprocess the grid once, then answer each rectangle query fast

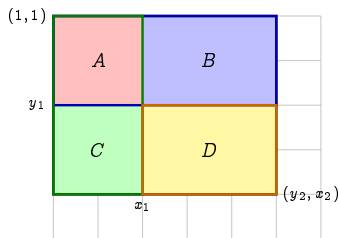


Why the 2D Formula Works



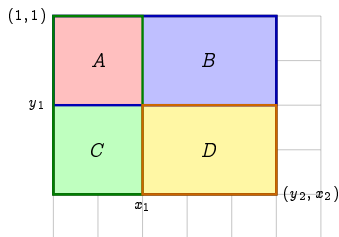
- Let D be the rectangle we want

Why the 2D Formula Works



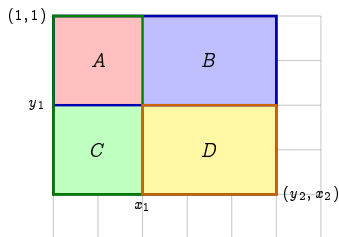
- Let D be the rectangle we want
- $pre[y_2][x_2]$ counts $A + B + C + D$

Why the 2D Formula Works



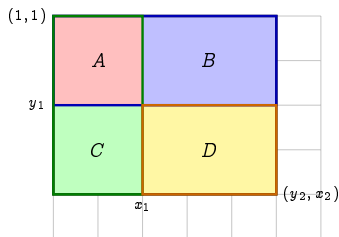
- Let D be the rectangle we want
- $pre[y_2][x_2]$ counts $A + B + C + D$
- Subtract the area above: $A + B$

Why the 2D Formula Works



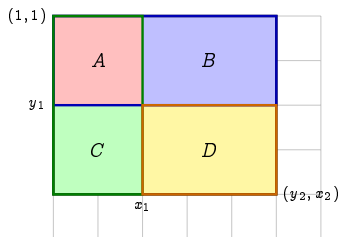
- Let D be the rectangle we want
- $pre[y_2][x_2]$ counts $A + B + C + D$
- Subtract the area above: $A + B$
- Subtract the area on the left: $A + C$

Why the 2D Formula Works



- Let D be the rectangle we want
 - $pre[y_2][x_2]$ counts $A + B + C + D$
 - Subtract the area above: $A + B$
 - Subtract the area on the left: $A + C$
- Now A was subtracted twice, so add it back once

Why the 2D Formula Works



- Let D be the rectangle we want
- $pre[y_2][x_2]$ counts $A + B + C + D$
- Subtract the area above: $A + B$
- Subtract the area on the left: $A + C$

- Now A was subtracted twice, so add it back once

$$D = (A + B + C + D) - (A + B) - (A + C) + A$$

- The four table lookups match the four prefix rectangles



Building the 2D Prefix Sum

- Define $grid[i][j] = 1$ if cell (i, j) has a tree, otherwise 0

Building the 2D Prefix Sum

- Define $grid[i][j] = 1$ if cell (i, j) has a tree, otherwise 0
- Define $pre[i][j]$ as the number of trees in the rectangle from $(1, 1)$ to (i, j)



Building the 2D Prefix Sum

- Define $grid[i][j] = 1$ if cell (i, j) has a tree, otherwise 0
- Define $pre[i][j]$ as the number of trees in the rectangle from $(1, 1)$ to (i, j)

$$pre[i][j] = grid[i][j] + pre[i - 1][j] \\ + pre[i][j - 1] - pre[i - 1][j - 1]$$



Building the 2D Prefix Sum

- Define $grid[i][j] = 1$ if cell (i, j) has a tree, otherwise 0
- Define $pre[i][j]$ as the number of trees in the rectangle from $(1, 1)$ to (i, j)

$$pre[i][j] = grid[i][j] + pre[i - 1][j] \\ + pre[i][j - 1] - pre[i - 1][j - 1]$$

- Building the table takes $\mathcal{O}(n^2)$



Answering a Forest Query

For a query rectangle (y_1, x_1) to (y_2, x_2) :

$$\begin{aligned}ans &= pre[y_2][x_2] \\&\quad - pre[y_1 - 1][x_2] \\&\quad - pre[y_2][x_1 - 1] \\&\quad + pre[y_1 - 1][x_1 - 1]\end{aligned}$$



Answering a Forest Query

For a query rectangle (y_1, x_1) to (y_2, x_2) :

$$\begin{aligned}ans &= pre[y_2][x_2] \\&\quad - pre[y_1 - 1][x_2] \\&\quad - pre[y_2][x_1 - 1] \\&\quad + pre[y_1 - 1][x_1 - 1]\end{aligned}$$

- Each query is answered in $\mathcal{O}(1)$



Answering a Forest Query

For a query rectangle (y_1, x_1) to (y_2, x_2) :

$$\begin{aligned}ans &= pre[y_2][x_2] \\&\quad - pre[y_1 - 1][x_2] \\&\quad - pre[y_2][x_1 - 1] \\&\quad + pre[y_1 - 1][x_1 - 1]\end{aligned}$$

- Each query is answered in $\mathcal{O}(1)$
- Total complexity becomes $\mathcal{O}(n^2 + q)$



Answering a Forest Query

For a query rectangle (y_1, x_1) to (y_2, x_2) :

$$\begin{aligned}ans &= pre[y_2][x_2] \\&\quad - pre[y_1 - 1][x_2] \\&\quad - pre[y_2][x_1 - 1] \\&\quad + pre[y_1 - 1][x_1 - 1]\end{aligned}$$

- Each query is answered in $\mathcal{O}(1)$
- Total complexity becomes $\mathcal{O}(n^2 + q)$
- This is the same pattern as 1D prefix sums: preprocess once, answer many queries fast



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

